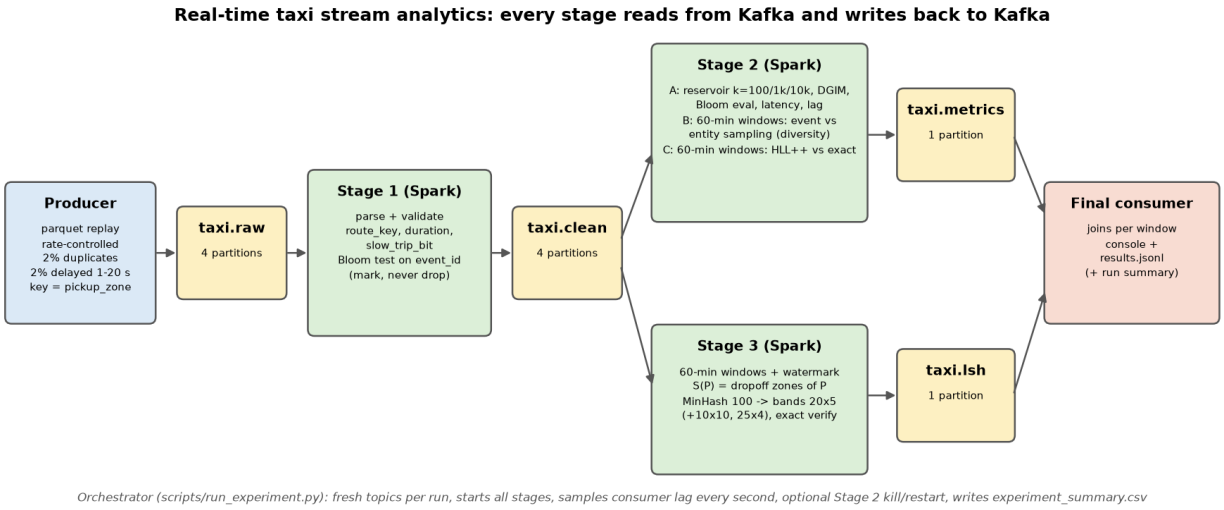
BIG DATA ANALYTICS

Assignment 2

Real-Time Taxi Stream Analytics

Apache Kafka + Spark Structured Streaming + Streaming Algorithms + MinHash-LSH



Implementation, code analysis and experimental results

Team: _____ Roll numbers: _____

Report generated 17 October 2026

1. Executive summary

This report documents a complete, reproducible streaming system for NYC yellow-taxi trips (January 2026, 3.72 million trips). A producer replays the trips as a live stream into Apache Kafka; three Spark Structured Streaming jobs then process the stream **exclusively through Kafka topics**: Stage 1 validates and enriches every event and runs a Bloom-filter duplicate test, Stage 2 computes reservoir samples, event- versus entity-sampling estimates, HyperLogLog distinct counts and DGIM counts against exact baselines, and Stage 3 finds pickup zones with similar destination sets using MinHash-LSH. A final Kafka consumer joins the outputs into the standardized *results.jsonl*.

Section 3 analyses the code component by component (with excerpts taken from the submitted source), section 4 presents the measurements of all required experiments and section 5 interprets them. The table below maps every requirement of the brief to its implementation and headline result.

Requirement (brief section)	Where	Result (default run unless stated)
Producer: controlled rate, 2 % duplicates, 2 % delayed (4.1)	producer/producer.py	1,018.5 ev/s achieved for 1 000 requested; 2057 duplicates, 1988 delayed events out of 100,000 trips
Stage 1 clean + enrich + Bloom (4.2, 5.1)	stage1_clean_bloom/stage1.py	99,738 accepted, 2,319 rejected; Bloom FP rate 3.1e-05, FN rate 0.0 (m = 1,917,012 bits, k = 7)
Reservoir sampling k = 100 / 1 000 / 10 000 (5.2)	stage2 query A, common/sketches.py	fare error 3.767 %, 0.299 %, 1.422 % for k = 100 / 1 000 / 10 000
Event vs entity sampling, non-linear statistic (5.3)	stage2 query B	mean absolute error 0.140 (event) vs 0.060 (entity) at equal storage
HyperLogLog distinct routes per window (5.4)	stage2 query C (Spark HLL++)	mean error 0.7339 %, max 1.732 % over 28 windows with a ~3.3 KB sketch
DGIM, k = 1 000 / 5 000 / 10 000 (5.5)	common/sketches.py DGIM (own code)	k=1000: 20.548 %, k=5000: 23.547 %, k=10000: 10.831 %; 14 buckets stored
MinHash-LSH, 20x5 + other settings (6)	stage3_lsh/stage3.py	20x5: recall 0.7418, candidate reduction 0.8734; 25x4 recall 0.9492, 10x10 recall 0.0305
Partition / consumer runs P1-P4 (7.2)	scripts/run_suite.py	Stage 2 throughput 5,055 / 5,257 / 5,324 / 4,842 ev/s at 6 000 ev/s input
Partition key K1 vs K2 (7.3)	partition_key config	busiest-partition share 0.300 (pickup_zone) vs 0.270 (no key)
Low / medium / high rate (7.4)	producer_rate config	p95 latency 4,112 / 6,229 / 9,334 ms; max raw lag 2,980 / 24,011 / 84,787
Late events with watermark (8.1)	Spark withWatermark(30 min)	1,429 events arrived after their window closed and were discarded
Restart of Stage 2 (8.2)	checkpoint + state snapshot	killed for 20 s; max taxi.clean lag 46,621; all events counted exactly once (99738 processed)
Standardized output (9)	final_consumer/final_consumer.py	results/results.jsonl, results/experiment_summary.csv, per-run folders

2. System architecture and environment

Figure 1 (title page) shows the pipeline. It follows the fixed topology of the brief: the producer is the only component that touches the source file, and every processing stage consumes one Kafka topic and publishes its output to another Kafka topic. Stage 2 and Stage 3 consume *taxi.clean* independently (separate Spark applications, separate offsets), so a slow or restarted Stage 2 does not affect Stage 3.

Topic	Partitions	Key	Producer -> consumer(s)	Content
taxi.raw	4 (1 in P1)	pickup_zone (none in K2)	producer -> Stage 1	raw trip events + EOS/WARMUP control
taxi.clean	4 (1 in P1)	pickup_zone (none in K2)	Stage 1 -> Stage 2, Stage 3	validated + enriched events, Bloom flag
taxi.metrics	1	-	Stage 2 -> final consumer	periodic summaries, per-window HLL and diversity
taxi.lsh	1	-	Stage 3 -> final consumer	per-window LSH metrics + candidate pairs

Machine: AMD Ryzen 7 7840HS (8 cores / 16 threads), 15.3 GB RAM, Windows 11 Home (build 26300), SSD. Docker and WSL were *not* available (they need administrator rights and a reboot), so Kafka runs natively on Windows; a *docker-compose.yml* is included for other machines. The broker, the producer, three Spark applications and the final consumer all share the laptop.

Component	Choice	Reason
Message broker	Apache Kafka 3.9.1, KRaft, 1 node	Mandatory. KRaft avoids ZooKeeper; one node is enough at laptop scale (replication factor 1).
Stream processing	Spark Structured Streaming 3.5.3 (PySpark), local[N]	Kafka source with checkpointed offsets, event-time windows and watermarks, built-in HLL++. local[N] gives N task slots = the 'parallel consumers' of section 7.
Sketches	own code in common/sketches.py	Bloom, reservoir, DGIM, MinHash + banding: every parameter is visible and reportable; DGIM must be group-implemented. Unit tests in tests/.
Kafka clients	confluent-kafka (librdkafka)	fast producer (>10 000 ev/s from Python), consumer for the final report, AdminClient for topics and lag.
Language	Python 3.12, Java 17 (Temurin)	PySpark 3.5 does not support Python 3.13+, pandas 3 or numpy 2; versions pinned in requirements.txt.

Repository layout

```
config.yaml                     single configuration file (all parameters of section 3 + run knobs)
producer/producer.py            parquet -> taxi.raw
stage1_clean_bloom/stage1.py    taxi.raw -> taxi.clean  (Spark)
stage2_stream_analytics/stage2.py taxi.clean -> taxi.metrics (Spark, 3 queries)
stage3_lsh/stage3.py            taxi.clean -> taxi.lsh  (Spark)
final_consumer/final_consumer.py taxi.metrics + taxi.lsh -> console + results.jsonl
common/  config.py  kafka_utils.py  spark_utils.py  sketches.py
scripts/ kafka_ctl.py  run_experiment.py  run_suite.py  analyze_results.py  show_lsh_window.py  build_report.py
tests/test_sketches.py          unit tests for Bloom, DGIM, reservoir, MinHash/LSH
results/ results.jsonl  experiment_summary.csv  analysis.md  plots/  runs/<run_id>/...
run_all.bat / run_all.sh / run_experiments.bat / setup_windows.ps1 / docker-compose.yml
```

3. Code analysis: how each component works

The code is organised exactly like the required submission structure: one folder per stage, plus a *common* package for configuration, Kafka/Spark helpers and the sketch implementations, and *scripts* for orchestration and analysis. All excerpts below are copied automatically from the source files when this report is generated.

3.1 Configuration and the single source of parameters

All parameters listed in section 3 of the brief (partitions, producer_rate, duplicate_rate, delayed_event_rate, max_delay_seconds, window_minutes, hll_precision, dgim_max_window, Bloom sizing, LSH settings, jaccard_threshold) live in *config.yaml*, together with run knobs (parallelism, partition_key, data segment, watermark, reservoir sizes, DGIM query sizes). Any value can be overridden per run with `--set key=value`. The orchestrator writes the effective configuration of a run to *results/runs/<run>/config.json* and passes its path to every process in the `BDA_CONFIG` environment variable, so all stages of a run see identical parameters and every result folder documents how it was produced.

common/config.py - load_config()

```
def load_config(path=None, overrides=None):
    path = path or os.environ.get("BDA_CONFIG") or ROOT / "config.yaml"
    path = Path(path)
    with open(path, encoding="utf-8") as f:
        cfg = json.load(f) if path.suffix == ".json" else yaml.safe_load(f)
    for item in overrides or []:
        key, _, value = item.partition("=")
        cfg[key.strip()] = _parse_value(value)
    for key in ("kafka_home", "hadoop_home", "work_dir"):
        cfg[key] = str(Path(os.path.expanduser(cfg[key])))
    if not Path(cfg["data_file"]).is_absolute():
        cfg["data_file"] = str(ROOT / cfg["data_file"])
    assert cfg["lsh_bands"] * cfg["lsh_rows_per_band"] == cfg["lsh_num_hashes"], \
        "lsh_bands * lsh_rows_per_band must equal lsh_num_hashes"
    return cfg
```

3.2 Producer (producer/producer.py)

The producer reads only the eight columns of section 2.1, keeps trips with pickup time from *segment_start* (Monday 2026-01-05 00:00) and replays the first *max_events* = 100 000 trips sorted by pickup time (ties broken by the original row number). Each trip becomes the JSON event of section 2.2; `event_id = 2026-01-row-<original parquet row>` is therefore stable, and *dropoff_time* is added because Stage 1 needs it for *trip_duration_min*. *producer_ts_ms* is set at the moment of sending (wall clock), *event_time* is the pickup time.

Rate control. The i-th source trip is due at `start + i / producer_rate`; the loop sleeps until then while releasing delayed events whose time has come. This keeps the rate accurate over the whole run instead of drifting with per-message overhead. **Fault injection.** With probability 0.02 a trip is held back in a heap for U(1, 20) seconds of wall clock (a *delayed* event), and with probability 0.02 a duplicate is scheduled 1-50 source events later with the same event_id.

producer/producer.py - pacing, delay and duplicate injection (main loop)

```
start = time.time()
last_print = start
for i, ev in enumerate(events):
    # pace: the i-th source event is due at start + i / rate
    due = start + i / rate
    while True:
        now = time.time()
        while delayed_heap and delayed_heap[0][0] <= now:
            _, _, dev = heapq.heappop(delayed_heap)
            send(dev)
        if now >= due:
            break
        producer.poll(0)
        time.sleep(min(0.002, due - now))
    while dup_pending and dup_pending[0][0] <= i:
        _, _, dev = heapq.heappop(dup_pending)
        send(dev)

    if rng.random() < cfg["delayed_event_rate"]:
        delay = rng.uniform(1, cfg["max_delay_seconds"])
        seq += 1
        heapq.heappush(delayed_heap, (time.time() + delay, seq, ev))
        delay_log.append((ev["event_id"], ev["event_time"], round(delay, 3)))
    else:
        send(ev)
    if rng.random() < cfg["duplicate_rate"]:
        seq += 1
        heapq.heappush(dup_pending, (i + rng.randint(1, 50), seq, ev))
        dup_log.append((ev["event_id"], ev["event_time"]))
```

The Kafka key is *pickup_zone* (baseline) or absent (`partition_key=none`, experiment K2, librdkafka's sticky partitioner). The producer measures what it really achieved: events per second for every wall-clock second, the replay-phase rate *actual_eps* and, separately, the rate including the tail spent releasing the last delayed events. Ground truth for the evaluation is written to the run folder (*producer_duplicates.csv*, *producer_delayed.csv*, *producer_stats.json*). Two kinds of control records complete the protocol: a short **WARMUP** burst before measurement (it makes Spark compile its query plans before the first real event; every stage ignores these records for all metrics) and an **EOS** (end of stream) record on every partition after the last event, whose far-future event time advances the watermarks so that the final windows close.

3.3 Stage 1: clean, enrich and Bloom filter (stage1_clean_bloom/stage1.py)

Stage 1 is a Spark Structured Streaming job that subscribes to *taxi.raw*. Parsing (from_json with a fixed schema), validation and the derived fields are Spark SQL expressions, so they run as parallel tasks over the Kafka partitions on the *local[N]* slots. A record is rejected (counted per reason, never forwarded) when a zone is missing or outside 1-265, a timestamp is unparsable, the duration is <= 0 or > 300 minutes, the fare is negative or > $1 000, or the distance is negative or > 100 miles. The last two limits were added after the first runs showed odometer values up to 269 097 miles in the source, which dominated the exact mean distance (see 4.2).

Stage 1 - Spark transformation (runs in parallel over Kafka partitions)

```
def build_query(spark, cfg):
    df = read_kafka(spark, cfg, TOPIC_RAW, RAW_SCHEMA)
    pickup = F.to_timestamp("event_time")
    dropoff = F.to_timestamp("dropoff_time")
    duration = (F.unix_timestamp(dropoff) - F.unix_timestamp(pickup)) / 60.0
    reason = (F.when(F.col("control").isNotNull(), F.lit(None))
              .when(F.col("pickup_zone").isNull() | F.col("dropoff_zone").isNull(), "missing_zone")
              .when(~F.col("pickup_zone").between(1, 265) | ~F.col("dropoff_zone").between(1, 265), "bad_zone")
              .when(pickup.isNull() | dropoff.isNull(), "bad_timestamp")
              .when((duration <= 0) | (duration > cfg["max_trip_minutes"]), "bad_duration")
              .when(F.col("fare_amount").isNull() | (F.col("fare_amount") < 0)
                    | (F.col("fare_amount") > cfg["max_fare"]), "bad_fare")
              .when(F.col("trip_distance").isNull() | (F.col("trip_distance") < 0)
                    | (F.col("trip_distance") > cfg["max_trip_miles"]), "bad_distance"))
    return (df.withColumn("reject_reason", reason)
              .withColumn("route_key", F.concat_ws("-", "pickup_zone", "dropoff_zone"))
              .withColumn("trip_duration_min", F.round(duration, 3))
              .withColumn("slow_trip_bit", (duration > cfg["slow_trip_minutes"]).cast("int")))
```

The micro-batch is then handed to `foreachBatch`, where the driver applies the Bloom filter to the events in arrival order and publishes one enriched event per accepted input to *taxi.clean* with the same key policy as the producer. One global filter (instead of one per task) gives a single consistent membership answer for the whole stream. The filter is sized from the configuration: $m = -n \ln p / (\ln 2)^2$ = 1 917 012 bits (234 KB) and $k = (m/n) \ln 2$ = 7 for n = 200 000 and p = 0.01; the k positions are derived from one 128-bit MurmurHash3 by double hashing.

Following the Bloom-filter rule of the brief, the filter never deletes anything: it only sets *bloom_seen_before*. An exact set of event ids (evaluation only) sets *gt_seen_before*, so every decision can be scored as TP / FP / FN / TN. Ground truth is "this event_id was already processed"; this is more precise than "the producer resent it", because a delayed original can arrive after its duplicate.

common/sketches.py - BloomFilter

```
class BloomFilter:
    """Standard Bloom filter sized from (expected items n, target false-positive rate p).

        m = -n ln p / (ln 2)^2      bits
        k = (m / n) ln 2            hash functions

    The k bit positions come from double hashing of one 128-bit MurmurHash3:
        g_i(x) = h1(x) + i * h2(x)  (mod m)
    """

    def __init__(self, expected_items, fp_rate):
        self.n_expected = int(expected_items)
        self.target_fp = float(fp_rate)
        self.m = max(8, int(math.ceil(-self.n_expected * math.log(self.target_fp) / math.log(2) ** 2)))
        self.k = max(1, int(round(self.m / self.n_expected * math.log(2))))
        self.bits = bytearray((self.m + 7) // 8)
        self.inserted = 0

    def _positions(self, key):
        h1, h2 = mmh3.hash64(key, signed=False)
        return [(h1 + i * h2) % self.m for i in range(self.k)]

    def test_and_add(self, key):
        """Return True if key was (probably) seen before, then insert it."""
        seen = True
        for pos in self._positions(key):
            byte, bit = divmod(pos, 8)
            mask = 1 << bit
            if not self.bits[byte] & mask:
                seen = False
                self.bits[byte] |= mask
        if not seen:
            self.inserted += 1
        return seen

    def __contains__(self, key):
        for pos in self._positions(key):
            byte, bit = divmod(pos, 8)
            if not self.bits[byte] & (1 << bit):
                return False
        return True

    def expected_fp_rate(self):
        """Theoretical FP probability after `inserted` distinct insertions."""
        return (1 - math.exp(-self.k * self.inserted / self.m)) ** self.k

    def params(self):
        return {"expected_items": self.n_expected, "target_fp": self.target_fp,
                "bit_array_size": self.m, "num_hashes": self.k, "inserted_ids": self.inserted,
                "memory_bytes": len(self.bits),
                "theoretical_fp_now": round(self.expected_fp_rate(), 6)}
```

Stage 1 foreachBatch - Bloom test vs exact ground truth (mark, never drop)

```
bloom_seen = self.bloom.test_and_add(row.event_id)
truly_seen = row.event_id in self.exact_seen
self.exact_seen.add(row.event_id)
if bloom_seen and truly_seen:
    st["bloom_tp"] += 1
elif bloom_seen:
    st["bloom_fp"] += 1
    if len(st["fp_examples"]) < 10:
        st["fp_examples"].append({"event_id": row.event_id, "event_time": row.event_time,
                                  "inserted_before": self.bloom.inserted - 1})
elif truly_seen:
    st["bloom_fn"] += 1
else:
    st["bloom_tn"] += 1
ev = {f: _clean(getattr(row, f), f in INT_FIELDS) for f in OUT_FIELDS}
ev["bloom_seen_before"] = int(bloom_seen)
ev["gt_seen_before"] = int(truly_seen)
ev["stage1_ts_ms"] = int(time.time() * 1000)
key = str(ev["pickup_zone"]).encode() if self.keyed else None
self._produce(json.dumps(ev).encode(), key)
st["accepted"] += 1
```

3.4 Stage 2: stream analytics (stage2_stream_analytics/stage2.py)

Stage 2 runs **three streaming queries in one Spark application**, all reading *taxi.clean*:

Query A (per event, foreachBatch in arrival order) maintains the order-dependent single-stream structures: three reservoirs, the DGIM counter, exact streaming means and the exact last-10 000 bits (baselines), Bloom evaluation counters, per-event latency, consumer lag and the late-event classification. Every *report_interval_seconds* (5 s) it publishes a compact *periodic* JSON summary to *taxi.metrics*. **Query B** groups by 60-minute event-time window and pickup zone (watermark 30 min) and keeps, per zone, the exact destination set, the event-sampled destination set and counts. **Query C** groups by window only and computes HyperLogLog++ next to the exact distinct count. B and C run in append mode: Spark emits a window exactly once, when the watermark passes its end, and the foreachBatch handlers publish *diversity* and *window* messages to *taxi.metrics*.

Stage 2 - the three streaming queries (main)

```
q_a = (clean.writeStream.foreachBatch(stage.process_events)
       .option("checkpointLocation", str(ckpt / "events")).queryName("events").start())

timed = (clean.withColumn("ts", F.to_timestamp("event_time"))
         .withWatermark("ts", f"{int(cfg['watermark_minutes'])} minutes"))
win = F.window("ts", f"{int(cfg['window_minutes'])} minutes")
rate_ppm = int(float(cfg["sampling_rate"]) * 1_000_000)
sampled = F.pmod(F.xxhash64("event_id"), F.lit(1_000_000)) < rate_ppm

diversity = (timed.withColumn("ev_sampled", sampled)
             .groupBy(win.alias("w"), "pickup_zone")
             .agg(F.count("*").alias("n"),
                  F.collect_set("dropoff_zone").alias("dsts"),
                  F.sum(F.col("ev_sampled").cast("int")).alias("n_sampled"),
                  F.collect_set(F.when(F.col("ev_sampled"), F.col("dropoff_zone"))).alias("sampled_dsts"))
             .select(F.col("w.start").alias("window_start"), F.col("w.end").alias("window_end"),
                     "pickup_zone", "n", "dsts", "n_sampled", "sampled_dsts"))
q_b = (diversity.writeStream.outputMode("append").foreachBatch(stage.process_diversity)
       .option("checkpointLocation", str(ckpt / "diversity")).queryName("diversity").start())

# No filter before the aggregation: Spark pushes filters below the watermark operator,
# which would hide the end-of-stream marker from it and leave the last windows open.
# The marker row has route_key = null, so count/approx_count_distinct/collect_set ignore it.
hll = (timed.groupBy(win.alias("w"))
       .agg(F.count("route_key").alias("events"),
            F.approx_count_distinct("route_key", hll_rsd(int(cfg["hll_precision"]))).alias("hll_estimate"),
            F.size(F.collect_set("route_key")).alias("exact_routes"))
       .select(F.col("w.start").alias("window_start"), F.col("w.end").alias("window_end"),
               "events", "hll_estimate", "exact_routes"))
q_c = (hll.writeStream.outputMode("append").foreachBatch(stage.process_hll)
       .option("checkpointLocation", str(ckpt / "hll")).queryName("hll").start())
```

Reservoir sampling (Algorithm R)

Algorithm R keeps the first k items, then replaces a random slot with probability k/n for the n-th item, which gives every item seen so far the same inclusion probability k/n. One reservoir per k (100, 1 000, 10 000) stores fare_amount and trip_distance together; the exact mean is a running sum over all events.

common/sketches.py - Reservoir

```
class Reservoir:
    """Algorithm R: uniform sample of k items from a stream of unknown length."""

    def __init__(self, k, n_fields, seed=0):
        self.k = int(k)
        self.items = np.zeros((self.k, n_fields), dtype=np.float64)
        self.n = 0
        self.rng = np.random.default_rng(seed)

    def add(self, row):
        if self.n < self.k:
            self.items[self.n] = row
        else:
            j = self.rng.integers(0, self.n + 1)
            if j < self.k:
                self.items[j] = row
        self.n += 1

    def mean(self, field):
        size = min(self.n, self.k)
        return float(self.items[:size, field].mean()) if size else float("nan")
```

DGIM (implemented by the group)

DGIM summarises the last N = 10 000 bits of *slow_trip_bit* (1 if the trip lasted more than 30 minutes). Each bucket stores the timestamp of its most recent 1 and a size that is a power of two; at most two buckets of each size exist, and when a third appears the two oldest of that size merge (keeping the newer timestamp). A bucket is discarded once its timestamp leaves the window. A query for the last k bits sums every bucket newer than t - k and counts only

half of the oldest one, because only that bucket can straddle the boundary; a size-1 bucket is exact. This bounds the relative error by 50 % while storing $O(\log^2 N)$ bits; in practice about 14 buckets are stored instead of 10 000 bits.

common/sketches.py - DGIM

```
class DGIM:
    """Datar-Gionis-Indyk-Motwani counter of 1s in the last N bits of a binary stream.

    Each bucket is (timestamp of its most recent 1, size), size a power of two.
    Invariant: at most two buckets of each size; when a third appears the two
    oldest of that size merge into one of double size (keeping the newer
    timestamp). Buckets whose timestamp falls out of the window are dropped.
    Memory: O(log^2 N) bits; relative error of a query <= 50%.
    """

    def __init__(self, window):
        self.N = int(window)
        self.t = 0                  # number of bits seen so far
        self.buckets = []           # newest first: [timestamp, size]

    def add(self, bit):
        self.t += 1
        # expire the oldest bucket once its most recent 1 left the window
        if self.buckets and self.buckets[-1][0] <= self.t - self.N:
            self.buckets.pop()
        if not bit:
            return
        self.buckets.insert(0, [self.t, 1])
        size = 1
        i = 0
        while True:
            # buckets of `size` are contiguous; find them starting at index i
            j = i
            while j < len(self.buckets) and self.buckets[j][1] == size:
                j += 1
            if j - i <= 2:
                break
            # three buckets of this size at i, i+1, i+2: merge the two oldest (i+1, i+2)
            newer_ts = self.buckets[i + 1][0]
            self.buckets[i + 1:i + 3] = [[newer_ts, size * 2]]
            i += 1
            size *= 2

    def query(self, k=None):
        """Estimate number of 1s among the last k bits (k <= N)."""
        k = self.N if k is None else min(int(k), self.N)
        total = 0
        last_size = 0
        for ts, size in self.buckets:
            if ts <= self.t - k:
                break
            total += size
            last_size = size
        if last_size == 0:
            return 0
        # only the oldest bucket may straddle the window boundary: count half of it.
        # A size-1 bucket is exactly its own timestamp, so it is fully inside.
        return total - last_size + (1 if last_size == 1 else last_size / 2)

    def num_buckets(self):
        return len(self.buckets)
```

HyperLogLog (Spark HLL++)

Distinct routes (route_key = pickup-dropoff) per window use Spark's built-in HyperLogLog++ (`approx_count_distinct`), which the brief allows. Spark derives the precision from the requested relative standard deviation as p = ceil(2 $\log_2$(1.106 / rsd)); the code inverts that formula so that *hll_precision* = 12 gives exactly 2^{12} = 4 096 six-bit registers (3 280 bytes) and a theoretical standard error of 1.04/sqrt(4096) = 1.6 %. The exact baseline is `size(collect_set(route_key))` in the same aggregation, so both see exactly the same events.

Event sampling versus entity sampling

The statistic is the fraction of active pickup zones with at least 10 distinct drop-off zones in a window. **Event sampling** keeps each event with probability 0.10, decided by a hash of event_id (deterministic, so a duplicate gets the same decision), and evaluates the property on the sampled events only. **Entity sampling** selects pickup zones by a hash of the zone id and keeps *all* events of the selected zones. To compare at the **same storage budget**, zones are taken in hash order until the number of their events would exceed the number of events the event sample stored in that window; a fixed-rate entity sample (hash below 10 %) is reported as well.

Stage 2 query B - event vs entity sampling with the same storage budget

```
# event sampling: Bernoulli(rate) on event_id (deterministic hash), property from sampled events only
es = g[g["n_sampled"] > 0]
ev_est = float((es["n_dst_sampled"] >= thr).mean()) if len(es) else 0.0
budget = int(g["n_sampled"].sum())

# entity sampling with the same storage budget: zones in hash order until the
# events kept (all events of each selected zone) would exceed the budget
g["h"] = g["pickup_zone"].apply(lambda z: mmh3.hash(str(int(z)), cfg["random_seed"], signed=False))
order = g.sort_values("h")
cum = order["n"].cumsum()
chosen = order[cum <= budget] if (cum <= budget).any() else order.head(1)
en_est = float((chosen["n_dst"] >= thr).mean())

# entity sampling at the same *rate* (fixed hash threshold), for reference
fixed = g[g["h"] < rate * 2 ** 32]
fx_est = float((fixed["n_dst"] >= thr).mean()) if len(fixed) else None
```

Late-event classification, latency, lag and restart safety

Latency is measured per event as Stage 2 processing time minus *producer_ts_ms* (p50/p95 over the run). **Lag** is computed from the offsets each stage has processed versus the log-end offsets; because Spark keeps its Kafka offsets in the checkpoint, Stages 1 and 2 also commit their positions to the consumer groups *stage1*/*stage2*, which the orchestrator samples every second. **Late events** are classified with the rule Spark applies: the watermark used for a batch is the maximum event time seen in earlier batches minus 30 minutes, and a row is discarded by a window aggregation when its window end is <= that watermark.

Restart safety. foreachBatch is at-least-once: after a crash Spark re-executes the last batch whose commit was not written. Query A therefore pickles its whole state together with the id of the last applied batch after every batch; on restart it reloads the snapshot and skips any batch id it has already applied, so no event is counted twice. Queries B and C keep their window state in Spark's checkpointed state store.

Stage 2 query A - idempotent replay check and state snapshot

```
if batch_id <= st.last_batch_id:
    st.replayed_batches_skipped += 1
    print(f"[stage2] batch {batch_id} already applied before restart -> skipped (no double counting)",
          flush=True)
    return
```

3.5 Stage 3: MinHash-LSH (stage3_lsh/stage3.py)

Stage 3 is a separate Spark application on *taxi.clean*. It groups by 60-minute window and pickup zone with the same 30-minute watermark and collects S(P), the set of drop-off zones. When a window closes, the driver keeps zones with at least 20 trips, builds a 100-value MinHash signature per set with the universal hash family $h_i(x) = (a_i x + b_i) \bmod (2^{31} - 1)$, applies banding (b bands x r rows; two signatures that agree on all r values of any band land in the same bucket and become a candidate pair), verifies candidates with exact Jaccard and returns pairs with J >= 0.50. For evaluation only, the exact all-pairs Jaccard gives the true similar pairs, from which recall, candidate precision, candidate reduction and both latencies are computed. The default 20x5 and two other 100-hash settings (10x10, 25x4) are evaluated on exactly the same sets, so their differences are caused only by the banding.

common/sketches.py - MinHash and banding

```
class MinHasher:
    """MinHash with universal hash family h_i(x) = (a_i x + b_i) mod p, p = 2^31 - 1."""

    def __init__(self, num_hashes, seed=1):
        rng = np.random.default_rng(seed)
        self.a = rng.integers(1, MERSENNE_P, size=num_hashes, dtype=np.int64)
        self.b = rng.integers(0, MERSENNE_P, size=num_hashes, dtype=np.int64)

    def signature(self, elements):
        x = np.fromiter(elements, dtype=np.int64)
        return ((self.a[:, None] * x[None, :] + self.b[:, None]) % MERSENNE_P).min(axis=1)


def lsh_candidates(signatures, bands, rows):
    """Banding: items sharing an identical band (r consecutive signature values)
    in at least one band become a candidate pair. Returns set of (p, q) with p < q."""
    candidates = set()
    for b in range(bands):
        buckets = defaultdict(list)
        lo, hi = b * rows, (b + 1) * rows
        for key, sig in signatures.items():
            buckets[(b, sig[lo:hi].tobytes())].append(key)
        for members in buckets.values():
            if len(members) > 1:
                candidates.update(combinations(sorted(members), 2))
    return candidates
```

stage3_lsh/stage3.py - per-window evaluation

```
# ---- ground truth: exact all-pairs Jaccard (evaluation only)
t0 = time.perf_counter()
exact = {}
for p, q in combinations(zones, 2):
    j = jaccard(sets[p], sets[q])
    if j >= self.threshold:
        exact[(p, q)] = j
exact_ms = (time.perf_counter() - t0) * 1000
true_pairs = set(exact)

results = []
default_pairs = []
for bands, rows in self.settings:
    t0 = time.perf_counter()
    sigs = {z: self.minhash.signature(sets[z]) for z in zones}
    t_sig = time.perf_counter()
    cands = lsh_candidates(sigs, bands, rows)
    t_band = time.perf_counter()
    verified = {}
    for p, q in cands:
        verified[(p, q)] = jaccard(sets[p], sets[q])
    returned = {pq for pq, j in verified.items() if j >= self.threshold}
    t_end = time.perf_counter()
    tp = len(returned & true_pairs)
    res = {
        "bands": bands, "rows": rows, "approx_threshold": round(lsh_threshold(bands, rows), 3),
        "candidates": len(cands), "returned_pairs": len(returned), "true_pairs": len(true_pairs),
        "true_in_candidates": tp,
        "recall": round(tp / len(true_pairs), 4) if true_pairs else None,
        "candidate_precision": round(tp / len(cands), 4) if cands else None,
        "candidate_reduction": round(1 - len(cands) / all_pairs, 4) if all_pairs else None,
        "lsh_latency_ms": round((t_end - t0) * 1000, 3),
        "signature_ms": round((t_sig - t0) * 1000, 3),
        "banding_ms": round((t_band - t_sig) * 1000, 3),
        "verify_ms": round((t_end - t_band) * 1000, 3),
```

3.6 Final consumer and standardized output

The final consumer subscribes to *taxi.metrics* and *taxi.lsh*. It keeps the latest periodic summary and, per window, the HLL, diversity and LSH messages; when all three of a window are present it appends one record with every mandatory field of section 9 (plus extra fields) to *results/results.jsonl* and the run's own file, and prints a one-line summary. After Stage 2 reports end of stream and no message arrives for 20 s, remaining windows are flushed and a *run_summary* record with run-level aggregates is written. Full LSH window messages (all candidate pairs with their exact Jaccard) are stored in *lsh_windows.jsonl* for the TA demonstration.

One record of results/runs/default/results.jsonl (nested by_k / settings lists shortened)

```
{
 "run_id": "default",
 "window_start": "2026-01-05T08:00:00",
 "window_end": "2026-01-05T09:00:00",
 "events_processed": 21664,
 "producer_eps": 989.0,
 "stage1_eps": 996.0,
 "stage2_eps": 653.7,
```

```
 "consumer_lag": 881,
 "latency_p50_ms": 3266,
 "latency_p95_ms": 4461,
 "bloom": {
  "fp_rate": 0.0,
  "fn_rate": 0.0,
  "false_positives": 0,
  "inserted_ids": 21201
 },
 "sampling": {
  "exact_mean_fare": 24.3708,
  "sample_mean_fare": 24.8986,
  "error_pct": 2.166,
  "k": 1000
 },
 "hll": {
  "exact_routes": 2484,
  "estimate": 2518,
  "error_pct": 1.369,
  "precision": 12,
  "registers": 4096,
  "relative_sd": 0.01625,
  "sketch_bytes": 3280,
  "exact_set_bytes_est": 178848
 },
 "dgim": {
  "k": 5000,
  "exact_ones": 752,
  "estimate": 801.0,
  "error_pct": 6.516,
  "buckets": 15,
  "N": 10000
 },
 "lsh": {
  "candidates": 266,
  "true_pairs": 156,
  "recall": 0.7051,
  "candidate_precision": 0.4135,
  "candidate_reduction": 0.807,
  "lsh_latency_ms": 5.57,
  "exact_latency_ms": 4.557,
  "active_zones": 53,
  "all_pairs": 1378
 },
 "events_in_window": 5602,
 "final_report_ts_ms": 1792227480545
}
```


3.7 Orchestration, Kafka control and Windows specifics

`scripts/run_experiment.py` runs one experiment end to end: it gives the run fresh topics, starts Stages 1-3 and the final consumer as separate processes, waits until every Spark query is running, starts the producer, samples consumer lag every second, optionally kills and restarts Stage 2, waits for the final consumer to finish and appends a row to *experiment_summary.csv*. `run_suite.py` runs all required experiments, `analyze_results.py` produces tables and plots.

Windows-specific engineering. (1) Deleting a topic makes a native Windows broker shut down (it cannot rename open segment files), so a clean run stops the broker, wipes its data directory and restarts it, a "clean equivalent" that the brief allows. (2) Spark needs *winutils.exe*, which links against the Visual C++ 2010 runtime; the Microsoft-signed DLL is extracted from the official redistributable without administrator rights. (3) Spark local mode occasionally fails to register its block manager at start-up; the stages bind the driver to 127.0.0.1, use a longer heartbeat interval, and the orchestrator relaunches a stage that does not become ready.

Two subtle Spark pitfalls were found and fixed during development. Filtering control rows with `where(...)` before a windowed aggregation is pushed by the optimizer *below* the watermark operator, which hides the EOS record from the watermark so the final windows never close; the aggregations therefore ignore control rows through null columns instead. And `collect()` converts timestamps to the machine's time zone (IST, +5:30) while `toPandas()` honours *spark.sql.session.timeZone = UTC*; mixing them misaligned the windows.

4. Results

All runs replay the same 100 000-trip segment (Monday 2026-01-05 00:00 to about 03:00 the next day, 28 event-time windows) on fresh topics. Tables 1 and 2 summarise all 12 runs; the following subsections analyse each requirement. Throughput of a stage is the number of events it processed divided by the time it needed for the first 95 % of them (this excludes the tail where only the last delayed events trickle in).

run	part.	slots	key	req./s	produced /s	stage1/s	stage2/s	p50 ms	p95 ms	max lag raw	max lag clean	skew raw
default	4	4	zone	1,000	1,018	1,019	995	2,840	4,346	2,955	4,091	0.304
P1	1	1	zone	6,000	6,049	5,800	5,055	5,878	7,502	24,277	36,740	1.000
P2	4	1	zone	6,000	6,050	5,648	5,257	7,482	10,322	27,858	49,886	0.300
P3	4	4	zone	6,000	6,050	5,875	5,324	4,193	5,551	24,111	24,767	0.297
P4	4	6	zone	6,000	6,050	5,425	4,842	7,247	9,408	45,312	27,081	0.296
K1	4	4	zone	6,000	6,050	5,361	4,869	6,698	8,819	39,803	32,155	0.300
K2	4	4	none	6,000	6,050	5,829	5,514	3,516	5,111	18,933	24,103	0.270
R-low	4	4	zone	1,000	1,018	1,019	993	2,824	4,112	2,980	4,134	0.302
R-medium	4	4	zone	6,000	6,049	5,844	5,389	4,665	6,229	24,011	23,544	0.297
R-high	4	4	zone	15,000	15,047	8,755	7,803	7,117	9,334	84,787	45,068	0.299
restart	4	4	zone	1,000	1,018	1,019	995	4,014	43,782	2,978	46,621	0.301
bloom-small	4	4	zone	3,000	3,043	3,030	2,874	3,239	4,832	9,650	12,765	0.301

Table 1 - performance of every run (results/experiment_summary.csv).

run	Bloom FP rate	FN	reservoir err % (k=1000)	DGIM err % (k=5000)	HLL mean err %	LSH recall 20x5	recall 10x10 / 25x4	diversity err event / entity
default	0.000031	0.0	0.30	23.55	0.734	0.742	0.030 / 0.949	0.140 / 0.060
P1	0.000031	0.0	2.40	30.11	0.728	0.738	0.031 / 0.949	0.140 / 0.060
P2	0.000031	0.0	0.81	30.53	0.748	0.738	0.030 / 0.951	0.140 / 0.060
P3	0.000031	0.0	0.80	30.53	0.741	0.738	0.031 / 0.950	0.141 / 0.060
P4	0.000031	0.0	0.04	30.53	0.735	0.741	0.030 / 0.951	0.140 / 0.060
K1	0.000031	0.0	1.40	30.96	0.727	0.741	0.030 / 0.950	0.140 / 0.060
K2	0.000031	0.0	1.34	30.11	0.752	0.738	0.031 / 0.950	0.140 / 0.059
R-low	0.000031	0.0	0.37	23.55	0.750	0.742	0.030 / 0.950	0.141 / 0.060
R-medium	0.000031	0.0	0.71	30.32	0.733	0.738	0.030 / 0.949	0.140 / 0.060
R-high	0.000031	0.0	0.97	30.53	0.743	0.740	0.031 / 0.950	0.141 / 0.060
restart	0.000031	0.0	3.09	23.36	0.772	0.741	0.030 / 0.950	0.141 / 0.062
bloom-small	0.926495	0.0	0.33	25.47	0.744	0.737	0.029 / 0.949	0.141 / 0.060

Table 2 - result quality of every run. Quality is essentially independent of the input rate and partitioning (see 4.6).

4.1 Bloom filter (5.1)

Default run: m = 1,917,012 bits (239,627 bytes), k = 7, 97,718 inserted ids. All 2,017 real repeats were flagged (2,017 true positives, FN = 0: a Bloom filter cannot produce false negatives because bits are never cleared). False positives: **3** (observed FP rate 3.1e-05). The theoretical FP probability at this fill level is $(1 - e^{-kn/m})^k$ = 0.000219: the filter was sized for 200 000 items but holds only ~100 000, so it operates far below its 1 % design point. The observed rate agrees with theory.

To demonstrate false positives clearly, the run **bloom-small** sizes the filter for 2 000 items (m = 19,171 bits, k = 7) on the same 100 000-event stream: it reports **90,538** false positives (FP rate 0.926495, theoretical 0.590015) and still 0 false negatives. Had Stage 1 dropped flagged events, this undersized filter would have silently deleted that many genuine trips, which is why the pipeline only marks them. Concrete false-positive examples (id never seen before, filter answered "seen"):


```
[
 {
  "event_id": "2026-01-row-00364524",
  "event_time": "2026-01-05T22:31:59",
  "inserted_before": 90781
 },
```

```
 {
  "event_id": "2026-01-row-00362923",
  "event_time": "2026-01-05T22:36:01",
  "inserted_before": 90983
 },
 {
  "event_id": "2026-01-row-02763898",
  "event_time": "2026-01-05T00:57:04",
  "inserted_before": 1641
 }
]
```


4.2 Reservoir sampling - a linear statistic (5.2)

k	sample size	exact mean fare	sample mean fare	error %	exact mean distance	sample mean distance	error %
100	100	22.069	21.237	3.77	4.039	3.452	14.54
1,000	1,000	22.069	22.135	0.30	4.039	4.031	0.21
10,000	10,000	22.069	22.382	1.42	4.039	4.075	0.88

The mean is a **linear** statistic: every sampled event is an unbiased estimate of it, so a uniform sample works well and the error shrinks roughly like the standard error sigma/sqrt(k). A single run is one random draw, so in an individual run a larger reservoir can occasionally be less accurate (as k = 10 000 versus k = 1 000 in the default run above); the reliable picture is the spread over the seven independent runs of Table 3. The fare error ranges over 0.51-13.03 % for k = 100, 0.30-2.40 % for k = 1 000 and 0.26-1.42 % for k = 10 000 (the stream holds 99,738 accepted events, so k = 10 000 is a 10 % sample): the worst case shrinks steadily with k. Fares and distances are heavy-tailed (airport trips), which is why k = 100 can be off by more than 10 %. Before the distance limit was added, the exact mean distance (6.70 miles) was dominated by a few bogus odometer values that a sample almost never contains, and every reservoir showed about 27 % error; this was a data-quality effect, not a sampling one.

run	k=100 fare / dist err %	k=1000 fare / dist err %	k=10000 fare / dist err %
default	3.77 / 14.54	0.30 / 0.21	1.42 / 0.88
R-low	8.95 / 11.87	0.37 / 1.13	0.76 / 0.76
R-medium	2.42 / 7.91	0.71 / 8.57	0.26 / 0.13
R-high	13.03 / 16.09	0.97 / 1.76	0.31 / 0.75
P1	0.51 / 4.64	2.40 / 0.60	0.87 / 0.66
P3	8.84 / 13.11	0.80 / 3.10	0.28 / 0.61
K2	2.31 / 1.53	1.34 / 1.03	0.66 / 0.77

Table 3 - reservoir error over independent runs: the variation between runs is the sampling noise itself (the reservoir RNG is seeded, but the arrival order differs between runs).

4.3 Event sampling versus entity sampling - a non-linear statistic (5.3)

window	active zones	exact	event sample	stored events	entity sample	stored events	zones kept
01-05 00:00	138	0.217	0.031	180	0.182	86	11
01-05 01:00	120	0.125	0.024	91	0.111	40	9
01-05 02:00	102	0.069	0.027	67	0.125	32	8
01-05 03:00	94	0.064	0.000	28	0.000	19	5
01-05 04:00	130	0.038	0.000	43	0.000	39	13
01-05 05:00	158	0.158	0.018	109	0.062	57	16
01-05 06:00	203	0.261	0.033	240	0.167	139	18
01-05 07:00	215	0.340	0.080	439	0.250	244	20
01-05 08:00	211	0.355	0.152	548	0.364	492	22
01-05 09:00	202	0.317	0.121	529	0.261	528	23
01-05 10:00	183	0.306	0.149	515	0.250	324	16
01-05 11:00	191	0.309	0.156	515	0.240	514	25
01-05 12:00	186	0.317	0.176	545	0.227	529	22
01-05 13:00	182	0.319	0.215	558	0.238	537	21
01-05 14:00	182	0.313	0.184	645	0.267	374	15
01-05 15:00	188	0.308	0.220	654	0.210	451	19

window	active zones	exact	event sample	stored events	entity sample	stored events	zones kept
01-05 16:00	187	0.326	0.215	696	0.353	372	17
01-05 17:00	168	0.363	0.284	704	0.385	353	13
01-05 18:00	166	0.343	0.282	682	0.263	648	19
01-05 19:00	155	0.342	0.231	531	0.308	345	13
01-05 20:00	140	0.386	0.182	425	0.400	331	10
01-05 21:00	135	0.370	0.210	418	0.400	243	10
01-05 22:00	135	0.385	0.113	358	0.333	329	12
01-05 23:00	147	0.306	0.043	245	0.100	73	10
01-06 00:00	142	0.268	0.044	150	0.333	77	9
01-06 01:00	109	0.165	0.026	81	0.250	68	8
01-06 02:00	98	0.092	0.045	42	0.143	29	7
01-06 03:00	68	0.015	0.000	13	0.000	8	5

Table 4 - fraction of active pickup zones with >= 10 distinct drop-off zones per 60-minute window, default run (10 % event sample; entity sample uses the same number of stored events).

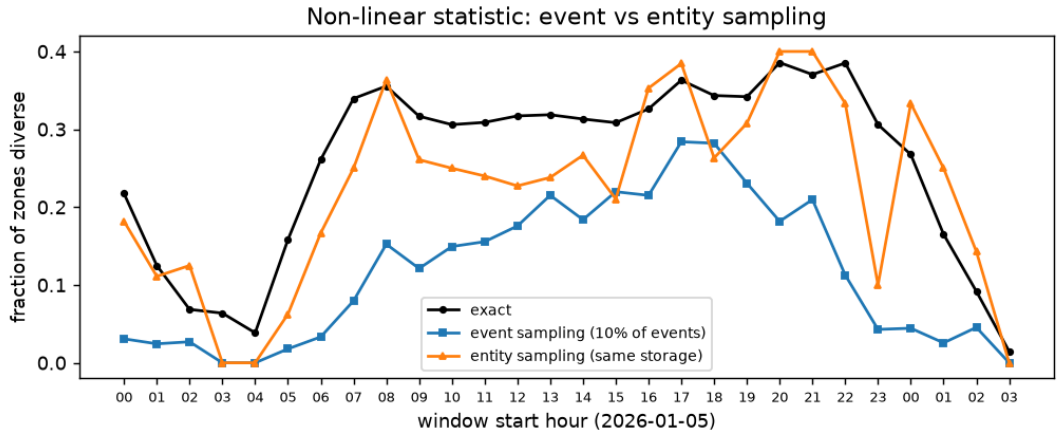


Figure 2 - exact fraction of diverse zones and the two sampling estimates per window.

Event sampling **systematically underestimates** the fraction of diverse zones (mean absolute error 0.140), while entity sampling at the same storage budget is unbiased and much closer (mean absolute error 0.060; fixed-rate entity sample 0.047). The reason is that the property "has trips to at least 10 distinct drop-off zones" depends on the *complete* behaviour of a zone. With 10 % event sampling, a zone with 40 trips to 15 destinations keeps about 4 trips and can show at most 4 destinations, so it can almost never qualify: the statistic is non-linear in the events and the sample distorts every zone. Entity sampling keeps all events of the selected zones, so each selected zone's property is evaluated exactly; the only error is which zones were selected, an ordinary sampling error that shrinks with the number of zones and is unbiased. Its weakness is visible too: with a small budget and skewed zones only a handful of zones fit (column "zones kept"), so individual windows are noisy. The general lesson: sample at the level of the unit the statistic is defined on.

4.4 HyperLogLog - distinct routes per window (5.4)

window	events	exact routes	HLL++ estimate	error %	exact set ~bytes	sketch bytes
01-05 00:00	1,744	1,135	1,119	1.41	81,720	3,280
01-05 01:00	879	677	670	1.03	48,744	3,280
01-05 02:00	528	431	428	0.70	31,032	3,280
01-05 03:00	347	314	317	0.95	22,608	3,280
01-05 04:00	453	397	397	0.00	28,584	3,280
01-05 05:00	1,028	792	791	0.13	57,024	3,280
01-05 06:00	2,466	1,619	1,614	0.31	116,568	3,280
01-05 07:00	4,513	2,405	2,434	1.21	173,160	3,280
01-05 08:00	5,602	2,484	2,518	1.37	178,848	3,280

window	events	exact routes	HLL++ estimate	error %	exact set ~bytes	sketch bytes
01-05 09:00	5,183	2,323	2,302	0.90	167,256	3,280
01-05 10:00	4,925	2,168	2,159	0.41	156,096	3,280
01-05 11:00	4,936	2,194	2,217	1.05	157,968	3,280
01-05 12:00	5,228	2,252	2,213	1.73	162,144	3,280
01-05 13:00	5,608	2,319	2,309	0.43	166,968	3,280
01-05 14:00	6,208	2,398	2,398	0.00	172,656	3,280
01-05 15:00	6,483	2,486	2,459	1.09	178,992	3,280
01-05 16:00	6,523	2,493	2,525	1.28	179,496	3,280
01-05 17:00	6,715	2,459	2,465	0.24	177,048	3,280
01-05 18:00	6,494	2,383	2,392	0.38	171,576	3,280
01-05 19:00	5,213	2,205	2,208	0.14	158,760	3,280
01-05 20:00	4,618	2,064	2,077	0.63	148,608	3,280
01-05 21:00	4,222	1,971	1,991	1.01	141,912	3,280
01-05 22:00	3,294	1,834	1,832	0.11	132,048	3,280
01-05 23:00	2,396	1,557	1,560	0.19	112,104	3,280
01-06 00:00	1,596	1,107	1,125	1.63	79,704	3,280
01-06 01:00	769	630	625	0.79	45,360	3,280
01-06 02:00	472	402	402	0.00	28,944	3,280
01-06 03:00	146	141	143	1.42	10,152	3,280

With p = 12 (4096 registers, theoretical relative standard error 0.01625), the mean error over 28 windows is **0.734 %** and the largest is 1.73 %, consistent with the 1.6 % standard error. The sketch needs about 3280 bytes regardless of the number of distinct routes, whereas the exact set needs one entry per route (about 72 bytes per route string in Python/JVM, roughly 180 KB for a busy hour); the sketch also merges across partitions by a register-wise maximum, which is what lets Spark compute it in parallel.

4.5 DGIM - recent slow trips (5.5)

run	k	exact 1s	DGIM estimate	abs error	error %	buckets
default	1,000	73	88.0	15.0	20.55	14
default	5,000	654	808	154	23.55	14
default	10,000	1,191	1,320	129	10.83	14
R-low	1,000	73	88.0	15.0	20.55	14
R-low	5,000	654	808	154	23.55	14
R-low	10,000	1,191	1,320	129	10.83	14
R-medium	1,000	112	136	24.0	21.43	14
R-medium	5,000	620	808	188	30.32	14
R-medium	10,000	1,251	1,320	69.0	5.52	14
R-high	1,000	116	136	20.0	17.24	14
R-high	5,000	619	808	189	30.53	14
R-high	10,000	1,290	1,320	30.0	2.33	14
P3	1,000	112	136	24.0	21.43	14
P3	5,000	619	808	189	30.53	14
P3	10,000	1,255	1,320	65.0	5.18	14

Table 6 - DGIM over slow_trip_bit (N = 10 000) at the end of each run.

Every estimate is within the 50 % guarantee, typically 5-25 % off, using about 14 buckets (a few hundred bits) instead of 10 000 bits. The error comes only from the oldest bucket, of which half is assumed to lie inside the window; it is largest when that bucket is big compared with the answer (k = 1 000 holds only ~100 slow trips, so a single 32- or 64-sized bucket at the boundary matters). The estimate depends only on the stream's bit sequence, which is why it does not change systematically with the input rate.

4.6 MinHash-LSH on destination sets (Task 3)

bands x rows	approx. threshold (1/b)^(1/r)	mean recall	mean candidate precision	mean candidate reduction	total candidates	total true pairs	mean LSH ms
20x5	0.549	0.742	0.293	0.873	3,537	1,769	3.01
10x10	0.794	0.030	0.708	0.997	86	1,769	1.94
25x4	0.447	0.949	0.190	0.740	7,122	1,769	3.53

Table 7 - LSH settings, default run, averaged over windows with at least one true pair (exact all-pairs baseline: 2.8096 ms per window on average).

window	zones (>=20 trips)	all pairs	true pairs	cand. 20x5	recall 20x5	cand. 25x4	recall 25x4	cand. 10x10	recall 10x10	LSH ms	exact ms
01-05 00:00	18	153	1	5	1.00	6	1.00	0	0.00	1.18	0.58
01-05 06:00	32	496	13	46	0.46	146	1.00	0	0.00	2.16	1.37
01-05 07:00	50	1225	131	196	0.63	467	0.95	7	0.04	4.55	3.90
01-05 08:00	53	1378	156	266	0.71	587	0.95	9	0.06	5.57	4.56
01-05 09:00	50	1225	165	273	0.75	555	0.92	6	0.03	5.82	5.84
01-05 10:00	45	990	95	176	0.63	375	0.91	3	0.03	4.58	4.72
01-05 11:00	45	990	107	189	0.62	405	0.96	3	0.02	4.10	4.05
01-05 12:00	46	1035	71	214	0.69	424	0.99	2	0.01	4.88	5.07
01-05 13:00	49	1176	138	241	0.77	451	0.96	13	0.09	6.55	5.58
01-05 14:00	49	1176	129	262	0.78	486	0.95	7	0.04	4.21	4.37
01-05 15:00	49	1176	120	271	0.75	449	0.93	2	0.00	4.19	4.54
01-05 16:00	50	1225	83	238	0.81	443	0.99	6	0.02	4.23	5.48
01-05 17:00	50	1225	144	294	0.76	539	0.97	7	0.03	5.77	6.79
01-05 18:00	49	1176	179	298	0.70	542	0.97	7	0.03	5.92	4.50
01-05 19:00	47	1081	83	173	0.80	347	0.89	8	0.08	3.69	4.07
01-05 20:00	44	946	79	151	0.76	315	0.95	2	0.03	3.46	3.74
01-05 21:00	43	903	52	153	0.83	318	1.00	3	0.02	3.72	3.34
01-05 22:00	39	741	21	70	0.67	201	0.76	1	0.05	2.56	2.99
01-05 23:00	31	465	2	15	1.00	55	1.00	0	0.00	2.80	2.08
01-06 00:00	20	190	0	5	-	8	-	0	-	1.12	0.63

Table 8 - per-window LSH results (windows with >= 10 active zones).

In busy hours about 45-53 zones qualify (>= 20 trips), i.e. 1 000-1 400 possible pairs, of which 80-180 are truly similar (J >= 0.5). The banding parameters behave exactly as the S-curve $P(\text{candidate}) = 1 - (1 - J^r)^b$ predicts. With 20x5 the threshold $(1/b)^{1/r} = 0.55$ lies slightly *above* 0.5, so pairs with J in [0.5, 0.6) are often missed: recall is about 0.74 while 87 % of pair comparisons are avoided. 25x4 (threshold 0.45) moves the curve left: recall rises to about 0.95 at the price of twice as many candidates (lower precision, smaller reduction) and slightly more time. 10x10 (threshold 0.79) generates almost no candidates and finds only the near-identical pairs (recall about 0.03). If high recall at J = 0.5 matters, 25x4 is the better choice for this workload.

With only ~50 sets of at most ~100 elements, the exact all-pairs computation takes about 2 ms, comparable to the LSH path (signatures + banding + verification, 2-4 ms). LSH does not pay off at this size; its benefit is that candidate generation grows roughly linearly with the number of sets while all-pairs grows quadratically, so it matters for thousands of zones or documents, not for 50.

Candidate pairs of window 2026-01-05T18:00:00 (20x5 banding; 298 candidates, 179 true pairs; first 14 by Jaccard, produced by scripts/show_lsh_window.py):

P	Q	zone P	zone Q	exact Jaccard	returned (J >= 0.5)
68	186	East Chelsea (Manhattan)	Penn Station/Madison Sq West (Manhattan)	0.662	yes
234	246	Union Sq (Manhattan)	West Chelsea/Hudson Yards (Manhattan)	0.656	yes
162	163	Midtown East (Manhattan)	Midtown North (Manhattan)	0.656	yes
107	163	Gramercy (Manhattan)	Midtown North (Manhattan)	0.654	yes
164	186	Midtown South (Manhattan)	Penn Station/Madison Sq West (Manhattan)	0.645	yes
48	163	Clinton East (Manhattan)	Midtown North (Manhattan)	0.643	yes
164	234	Midtown South (Manhattan)	Union Sq (Manhattan)	0.632	yes
68	161	East Chelsea (Manhattan)	Midtown Center (Manhattan)	0.630	yes
186	234	Penn Station/Madison Sq West (Manhattan)	Union Sq (Manhattan)	0.627	yes
68	234	East Chelsea (Manhattan)	Union Sq (Manhattan)	0.627	yes
161	162	Midtown Center (Manhattan)	Midtown East (Manhattan)	0.623	yes
100	107	Garment District (Manhattan)	Gramercy (Manhattan)	0.620	yes
186	246	Penn Station/Madison Sq West (Manhattan)	West Chelsea/Hudson Yards (Manhattan)	0.617	yes
141	229	Lenox Hill West (Manhattan)	Sutton Place/Turtle Bay North (Manhattan)	0.615	yes

4.7 Partitions and consumers: P1-P4 (7.2)

run	partitions	slots	produced/s	Stage 1/s	Stage 2/s	p50 ms	p95 ms	max lag raw	max lag clean
P1	1	1	6,049	5,800	5,055	5,878	7,502	24,277	36,740
P2	4	1	6,050	5,648	5,257	7,482	10,322	27,858	49,886
P3	4	4	6,050	5,875	5,324	4,193	5,551	24,111	24,767
P4	4	6	6,050	5,425	4,842	7,247	9,408	45,312	27,081

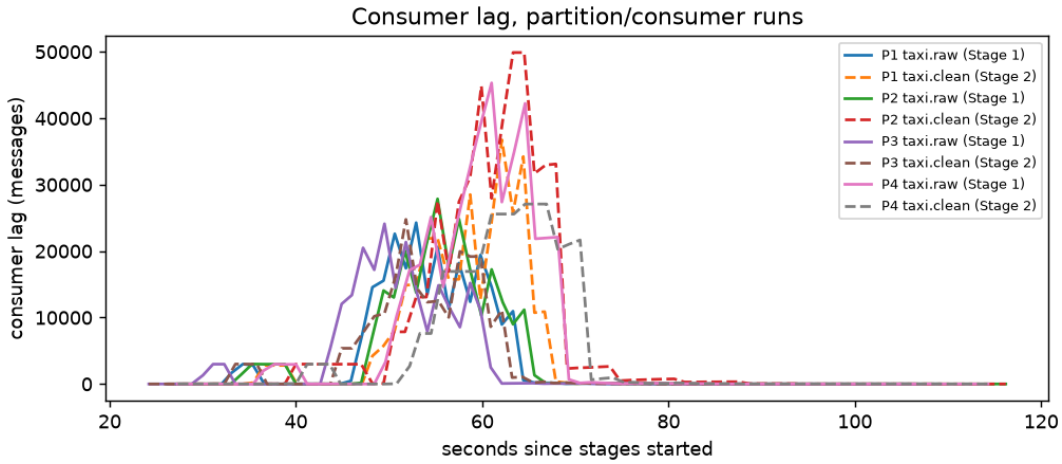


Figure 3 - consumer lag during the P1-P4 runs.

All four runs use the same segment at 6 000 ev/s (the medium rate, close to capacity), so differences come from the partition/slot layout. Stage 2 throughput was 5,055 (P1), 5,257 (P2), 5,324 (P3) and 4,842 (P4) events/s; Stage 1 reached 5,800 / 5,648 / 5,875 / 5,425 events/s. In other words, **more partitions and more slots did not materially increase throughput on this setup**, and the differences between runs are of the same size as the run-to-run noise.

Why P2 versus P3 shows little gain

With one slot (P2), Spark runs the four Kafka-partition tasks of a micro-batch one after another; with four slots (P3) they run in parallel. But only part of each micro-batch is parallel: reading and parsing the records. The per-batch logs show that the other part, which runs on the single driver (collecting the batch, the Bloom test in arrival order, JSON serialisation and publishing to taxi.clean in Stage 1; the reservoir/DGIM loop in Stage 2), takes as long as the parallel part. By Amdahl's law, speeding up roughly half of the work 4x yields well under 2x, and Spark's fixed per-micro-batch cost (planning, writing the offset and commit logs of the checkpoint) is paid regardless of the slot count. In addition, three Spark applications, Kafka and the producer share 16 hardware threads, so extra slots in

one application compete with the others. The order-dependent sketches are deliberately global (one Bloom filter, one DGIM stream), which is exactly what limits scaling; a scalable design would partition them by key (e.g. one Bloom filter per Kafka partition, which works because duplicates share the pickup_zone key).

Why 6 consumers do not beat 4 on a 4-partition topic (P4)

In a Kafka consumer group each partition is assigned to **at most one** consumer, so with 4 partitions at most 4 consumers receive data and the remaining ones sit idle as hot standbys. Spark's Kafka source behaves the same way by default: it creates one task per topic partition per micro-batch, so with local[6] two of the six slots have nothing to do. P4 therefore cannot be faster than P3 (here 4,842 vs 5,324 events/s in Stage 2, equal within noise, with slightly more scheduling overhead). Parallelism beyond the partition count requires more partitions, or splitting partitions by offset ranges (Spark's minPartitions option), which gives up per-partition ordering.

P1 also shows the cost of a single partition: all 105 058 raw messages go to partition 0 (skew 1.0). It was not slower here because the bottleneck is the serial driver work, not the Kafka partition; with a heavier parallel part, one partition would cap the parallelism at one task.

4.8 Partition key: pickup_zone versus no key (7.3)

run	key	msgs p0	msgs p1	msgs p2	msgs p3	skew (busiest share)	Stage 1/s	Stage 2/s	p50 ms	p95 ms
K1	pickup_zone	21,885	31,531	29,372	22,273	0.300	5,361	4,869	6,698	8,819
K2	none	25,877	25,691	25,129	28,364	0.270	5,829	5,514	3,516	5,111

Skew. With key = pickup_zone the busiest partition carries 0.300 of the messages (ideal 0.25). Kafka assigns a key to partition murmur2(key) mod 4, so all trips of a zone land in one partition. Demand is very uneven: the three busiest zones (Upper East Side North/South, Midtown Center) each account for about 4 % of all trips, and how the ~260 zones happen to hash decides which partition becomes hot. Without a key (K2), librdkafka's sticky partitioner fills one partition per batch and rotates, which gives a near-uniform split (busiest share 0.270).

Throughput. Stage 1 / Stage 2 processed 5,361 / 4,869 ev/s with the key and 5,829 / 5,514 ev/s without it (p95 latency 8,819 vs 5,111 ms). The keyless run was somewhat faster, but by about as much as two runs with *identical* configuration differ (P3 and K1 are the same setup and reached 5,324 and 4,869 ev/s), so the effect of skew is within run-to-run noise here: the partition is not the bottleneck (see 4.7). With a parallel bottleneck, the hot partition would set the pace of every micro-batch (a straggler task).

Repartitioning. Spark does not know how Kafka partitioned the data, so every per-pickup-zone state (the window aggregations of Stage 2 query B and Stage 3) **always** goes through a shuffle (hash-partitioning by pickup_zone into spark.sql.shuffle.partitions), with or without the Kafka key. The key therefore buys no locality in Spark; what it does buy is per-zone ordering within a partition (all events of a zone are consumed in production order) and the possibility of a shuffle-free, per-partition consumer such as Kafka Streams or a partition-local Bloom filter. The trade-off is ordering and locality versus load balance; in our measured workload balance mattered little, so the key's value lies mainly in ordering guarantees.

4.9 Producer rate: low / medium / high (7.4)

Rate	Produced/s	Processed/s	p50 latency	p95 latency	max lag	HLL err.	DGIM err.	LSH recall
low (1,000/s)	1,018	993	2,824 ms	4,112 ms	2,980 / 4,134	0.750 %	23.55 %	0.742
medium (6,000/s)	6,049	5,389	4,665 ms	6,229 ms	24,011 / 23,544	0.733 %	30.32 %	0.738
high (15,000/s)	15,047	7,803	7,117 ms	9,334 ms	84,787 / 45,068	0.743 %	30.53 %	0.740

Table 11 - the table of section 7.4 (max lag = taxi.raw / taxi.clean; HLL error = mean over windows; DGIM error at k = 5000).

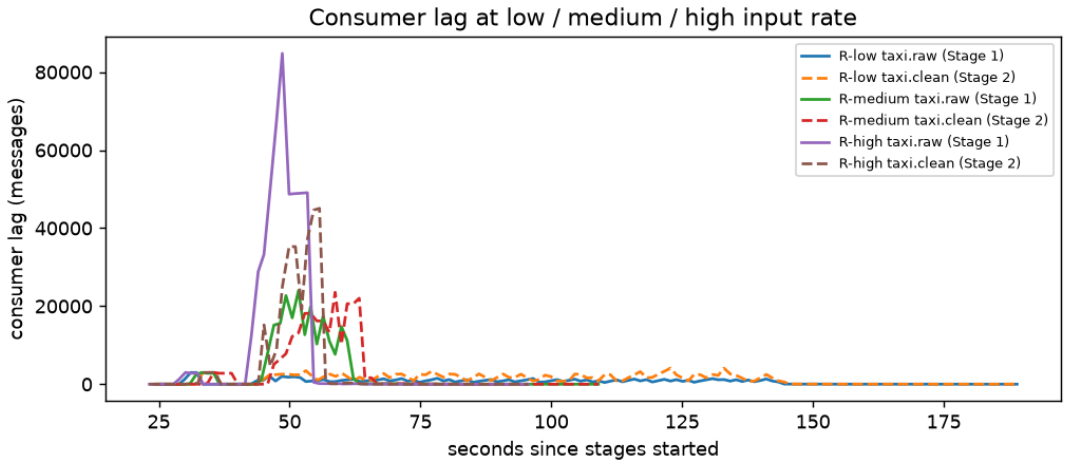


Figure 4 - consumer lag at the three input rates.

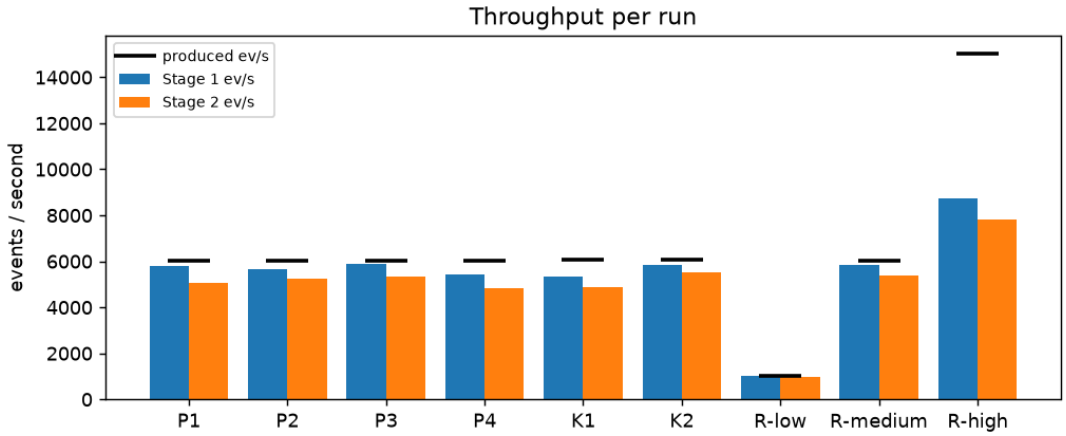


Figure 5 - produced versus processed throughput per run.

The capacity of this setup was first calibrated at 10 000 ev/s (Stage 1 about 8 000 ev/s, Stage 2 about 7 200 ev/s). Accordingly: **low** = 1 000 ev/s, **medium** = 6 000 ev/s (just below capacity), **high** = 15 000 ev/s (about twice the capacity).

Low (1,018 ev/s). The pipeline keeps up throughout: lag stays at the size of one or two micro-batches and drains immediately. Latency (p50 2,824 ms, p95 4,112 ms) is not caused by queueing but by the *micro-batch floor*: each Spark stage starts a new batch only about every 1-1.5 s because of its fixed per-batch cost (planning plus writing offset and commit logs to the checkpoint, which on Windows goes through winutils), and an event crosses two Spark stages.

Medium (6,049 ev/s). Stage 2 processed 5,389 ev/s. Lag rises to about 24,011 (raw) / 23,544 (clean) during the replay, then drains within a few seconds after the producer stops; micro-batches grow with the backlog, which amortises the fixed batch cost and keeps the system stable. Latency is higher (p50 4,665, p95 6,229 ms).

High (15,047 ev/s). The producer exceeds the processing capacity (7,803 ev/s in Stage 2): lag grows during the whole replay, to 84,787 messages on taxi.raw and 45,068 on taxi.clean, and latency rises to p50 7,117 / p95 9,334 ms. Because Kafka buffers the excess durably, nothing is lost: all events are processed once the producer stops.

Result quality does not degrade with speed: HLL error 0.750 / 0.733 / 0.743 % and LSH recall 0.742 / 0.738 / 0.740 for low / medium / high. The sketches see the same events regardless of how fast they arrive; lag changes *when* an answer is available, not the answer. The one mechanism through which speed does change results is event time: a delay of up to 20 s of wall clock corresponds to more event time at a higher replay rate, so more of the deliberately delayed events miss their window (1,417 / 1,790 / 1,793 events classified as dropped). Differences in DGIM and reservoir error between runs come from the arrival order, which differs slightly from run to run (four partitions are interleaved by the micro-batching), not from the rate.

4.10 Event time and delayed events (8.1)

Policy. Stages 2 and 3 use 60-minute tumbling event-time windows with `withWatermark(event_time, "30 minutes")` in append mode. A window's result is emitted once, when the watermark (maximum event time seen minus 30 minutes) passes the window end. Rows that arrive for a window that has already been emitted are discarded; results are never updated afterwards. Spark guarantees to include every row that is less than 30 minutes (event time) late; later rows may or may not be dropped, depending on micro-batch boundaries.

Scale of the delay. At 1 000 ev/s the replay covers about 5 700 trips per event-time hour, so a producer delay of 1-20 s of wall clock means 10 minutes to 3.5 hours of event time. In the default run the producer delayed 1988 events. Joining its ground truth with Stage 2's log of dropped ids: **1429** of them arrived after their window was closed and were **discarded**, 559 were still **included** because their window was still open. The wall-clock delay alone does not decide the outcome: the longest delay still included was 19.8 s and the shortest delay that was dropped only 1.2 s. What matters is how much *event time* the stream advanced during the delay and where in its window the event falls: at night a second of replay covers far more event time (fewer trips per hour) than at rush hour, and an event from the end of a window has 30 minutes of slack less than one from its start. 0 dropped events were not deliberately delayed, i.e. the reordering caused by interleaving four partitions and micro-batches stayed well inside the 30-minute watermark.

Stage 2's classification of all events: 27,431 in order, 70,878 out of order but inside an open window (lateness buckets {'<1min': 67335, '1-10min': 2810, '10-30min': 407, '>=30min': 326}), 1,429 beyond the watermark. Spark's own counter numRowsDroppedByWatermark reported {'diversity': 967, 'hll': 151}. The numbers differ between the queries because each query has its own micro-batches and therefore its own watermark progression: this is exactly the "may or may not be dropped" part of the watermark contract. Two examples recorded by Stage 2 (one discarded, one included although it was more than 10 minutes late):


```
[
 {
  "class": "late_dropped",
  "event_id": "2026-01-row-00291067",
  "event_time": "2026-01-05T01:56:25",
  "max_event_time_seen": "2026-01-05T04:33:04",
  "lateness_min": 156.65,
  "window_end": "2026-01-05T02:00:00",
  "watermark": "2026-01-05T03:08:33"
 },
 {
  "class": "late_included",
  "event_id": "2026-01-row-00290330",
  "event_time": "2026-01-05T00:53:28",
  "max_event_time_seen": "2026-01-05T02:04:54",
  "lateness_min": 71.43,
  "window_end": "2026-01-05T01:00:00",
  "watermark": "2026-01-04T23:58:26"
 }
]
```


4.11 Restart and recovery of Stage 2 (8.2)

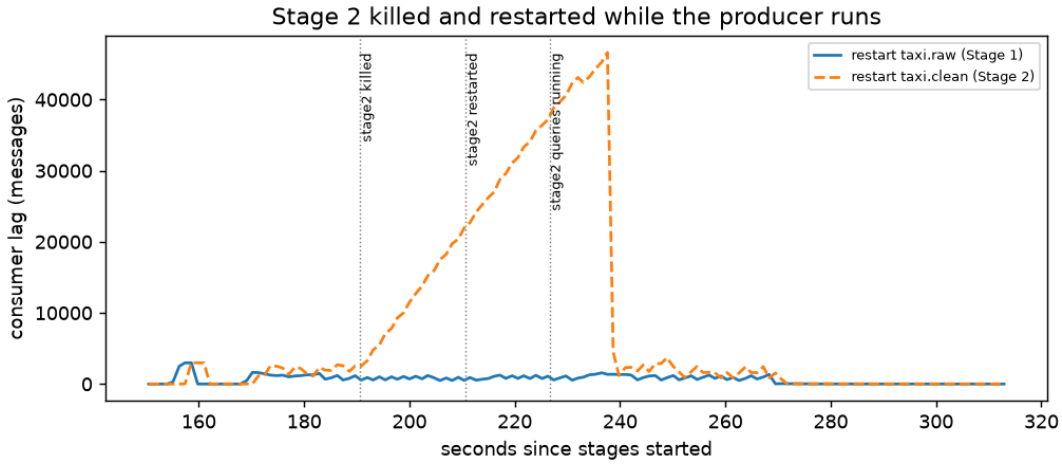


Figure 6 - lag of taxi.clean while Stage 2 is killed and restarted (producer keeps running).

The run **restart** (1 000 ev/s) kills the Stage 2 process tree (hard kill, no clean shutdown) 40 s after the producer started and relaunches it 20 s later, while the producer, Stage 1 and Stage 3 keep running. Stage 2 was killed at t = 190.5 s and its queries were running again at t = 226.6 s (Spark start-up adds to the 20 s outage).

Kafka lag and offsets. During the outage nothing consumes taxi.clean, so its lag grows linearly with the input (Figure 6, up to 46,621 messages) while taxi.raw lag stays near zero. Kafka retains the messages, so nothing is lost. On restart, Spark reads the last committed offsets of every query from its checkpoint and continues from there; the large first micro-batch drains the backlog in a few seconds and lag returns to the normal level. Latency of the events produced during the outage includes the outage (p95 43,782 ms).

Application state. The window state of queries B and C is restored by Spark from the checkpointed state store. Query A's Python state (reservoirs, DGIM, exact baselines, latency samples, counters) was restored from its snapshot: `restored state after batch 15: 18987 events`. Without the snapshot, the restarted Stage 2 would have resumed the offsets but started all sketches from zero.

Duplicate processing. Spark's foreachBatch sink is at-least-once: if the process dies after a batch was applied but before Spark wrote its commit, that batch is executed again after the restart with the same batch id; the batch-id check skips such a batch. In this run the kill landed after Spark had committed the last applied batch, so no batch was re-executed and the check did not have to fire. The end result confirms exactly-once state: 99,738 events were counted, the same as in the uninterrupted default run (99,738), and all 28 windows were reported (mean HLL error 0.7716 %). Messages published to taxi.metrics during a replayed batch could still be duplicated (Kafka output of foreachBatch is at-least-once); the final consumer is idempotent per window (the latest message wins).

5. Discussion

Approximation versus state. Every sketch delivered its accuracy at a small, fixed memory cost: HLL++ about 1 % error with 3.3 KB per window instead of a set of ~2 500 routes; DGIM 5-30 % error with ~14 buckets instead of 10 000 bits; a 234 KB Bloom filter with essentially no false positives at half load. The exact baselines were affordable here only because the stream is small; their state grows with the data, the sketches' does not.

Sampling design must follow the statistic. The mean fare (linear) is estimated well by any uniform sample, and the error behaves like 1/sqrt(k). The fraction of diverse zones (non-linear, defined per entity) is biased under event sampling and only becomes reliable when whole entities are sampled.

Parallelism is bounded by partitions and by the serial part. Consumers beyond the partition count stay idle, and adding slots helps only the parallel part of the work. Our pipeline is dominated by driver-side, order-dependent sketch updates and Spark's per-batch overhead, so 1, 4 and 6 slots performed alike. More partitions are not automatically better: they add overhead, and keyed partitions can be skewed (30 % instead of 25 % here).

Speed affects timeliness, not correctness, with one exception. A higher producer rate increased lag and latency but not sketch error or LSH recall, and Kafka absorbed every burst. The exception is event time: when late data is dropped by a watermark, the amount of data that is "late" depends on how event time and processing time relate, which changes with the rate.

LSH is a candidate filter. The banding parameters set the similarity threshold of the S-curve: 20x5 (0.55) misses part of the pairs near J = 0.5, 25x4 (0.45) finds 95 % of them with twice the candidates, 10x10 (0.79) finds almost nothing at this threshold. At ~50 zones per window exact all-pairs is just as fast; LSH's linear candidate generation pays off only at much larger scales.

6. Limitations and honest notes

Laptop scale and short runs. 100 000 events last 100 s at 1 000 ev/s but only 17 s at 6 000 and 7 s at 15 000 ev/s, so the medium/high runs are dominated by a burst and its drain rather than a long steady state. All processes share one machine, so measurements include interference (run-to-run noise of about 5-10 %).

Driver-side sketches. Bloom, reservoir and DGIM run on the Spark driver in foreachBatch to keep one global, order-dependent structure. This is simple and exactly reportable but limits horizontal scaling; a partitioned design (per-key sketches merged at report time) would scale better.

Latency floor. Spark micro-batches on Windows cost about 1 s each (checkpoint writes via winutils), which dominates latency at low rates. Continuous processing or Kafka Streams would reach lower latency.

Clean runs by broker reset. Topic deletion crashes native Kafka on Windows, so native runs restart the single-node broker with an empty log directory; on Docker/Linux topics are deleted and recreated.

Known issues. Spark local mode sometimes fails to start (block-manager registration race); the orchestrator relaunches such a stage, and the runs that failed in the first suite were rerun. The run *bloom-small* reports a long duration because the laptop slept during its shutdown phase; its measurements were taken before that and are complete.

Event time. Results are emitted once per window (append mode) and never updated; a late event beyond the watermark is discarded rather than triggering a corrected result.

7. Reproducibility and submission checklist

Checklist item (brief section 14)	Status / evidence
Producer reads taxi data and writes only to taxi.raw	producer/producer.py (the only file reader)
Stage 1 reads taxi.raw and writes taxi.clean	stage1_clean_bloom/stage1.py
Stage 2 reads taxi.clean and writes taxi.metrics	stage2_stream_analytics/stage2.py
Stage 3 reads taxi.clean and writes taxi.lsh	stage3_lsh/stage3.py
Final consumer reads Kafka output topics and creates results.jsonl	final_consumer/final_consumer.py
Bloom false-positive rate is measured	4.1 (default + bloom-small with examples)
Reservoir sampling and non-linear sampling comparison complete	4.2, 4.3
HLL/FM exact-vs-estimate result present	4.4 (per window)
DGIM answers k = 1 000 / 5 000 / 10 000	4.5
LSH recall and candidate reduction reported	4.6 (20x5, 10x10, 25x4)
P1-P4 partition/consumer experiments present	4.7
pickup_zone vs round-robin comparison present	4.8
Low/medium/high producer-rate experiment present	4.9
p50/p95 latency and Kafka lag reported	Table 1, Figures 3, 4, 6
Late event and restart behaviour demonstrated	4.10, 4.11
README with one-command default run	README.md, run_all.bat / run_all.sh
LLM_USAGE.md included	LLM_USAGE.md